The NEVPT2 nuclear gradient: SC and PC, on CASCI and CASSCF orbitals

`nevpt2-gpu-demo`, `docs/nuclear_gradients.tex`

Abstract

The tree assembles the analytic nuclear gradient of the strongly-contracted (SC) and, with `--pc`, the partially-contracted (PC) NEVPT2 energy, and checks it against a numerical gradient of re-converged energies to $10^{-6}\,E_h/a_0$ per component. The work is split on purpose. The GPU (`nevpt2_sc_pseudodensity --dump-source-terms`) computes the two *source terms*, the energy's derivatives with respect to the orbital rotations and the CI coefficients, in reverse mode, and writes them out. Offline Python (`reference_data/nevpt2_grad.py`) reads them, solves the Lagrangian's Z-vector equations, and takes the nuclear derivative of one scalar along a frozen orthonormal path. A second offline script (`nevpt2_numgrad.py`) supplies the numerical gradient that the result is gated against. This document covers the four combinations $\{\mathrm{SC}, \mathrm{PC}\} \times \{\mathrm{CASCI}, \mathrm{CASSCF}\}$ and the density-fitted SC case. For each it gives the equations the code evaluates and pseudocode for the path through the tree. The derivation is `docs/gradient-theory.md`, the GPU side in prose is `docs/gradients.md`, and the energy itself is `docs/energy.tex`. Like every file in `docs/`, this one states what the code *does*, and it carries no measured numbers.

Contents

1 The four paths, and where they actually differ **2**

2 Notation and conventions **3**

3 What is differentiated **4**

4 Shared stage A: the source terms Y and y **5**
4.1 What the Lagrangian reads 5
4.2 Provider 1: the GPU, by reverse mode 5
4.3 Provider 2: offline central differences 7

5 Shared stage B: the frozen orthonormal path, and Φ **7**

6 The CASCI-branch Lagrangian and its solve **8**
6.1 The constraints 8
6.2 The stationarity equations 9

7 The CASSCF-branch Lagrangian and its solve **10**

8 The numerical gradient, and the gate **11**

9 Path 1: SC-NEVPT2 on a CASCI reference **12**
9.1 The density-fitted variant 12

10 Path 2: PC-NEVPT2 on a CASCI reference **13**

11 Path 3: SC-NEVPT2 on a CASSCF reference **14**

12 Path 4: PC-NEVPT2 on a CASSCF reference **14**

13 Checks that read no residual **15**

14 Scope and limits **16**

A The source-term inventory **17**

B The pseudocanonical multipliers z^c **17**

1 The four paths, and where they actually differ

	CASCI orbitals (RHF)	CASSCF orbitals
SC	§9 (fitted: §9.1)	§11
PC (`--pc`)	§10	§12

Table 1: The four paths, and the section that walks through each.

For the energy the two axes of Table 1 split the work at the input and at the class energies (`docs/energy.tex`, §1). For the gradient they split it differently:

- **CASCI vs. CASSCF changes the Lagrangian.** The energy is the same functional on both kinds of orbitals. What differs is how the orbitals were made, and that is what the orbital constraints state. On RHF orbitals the core/active/virtual partition holds because the RHF Fock matrix is block-diagonal over it (§6). On CASSCF orbitals it holds because the CASSCF orbital gradient vanishes (§7). The first gives a CI equation decoupled from the orbital one. The second gives one coupled CP-CASSCF system. The golden's header field `reference_kind` records which branch applies, and `nevpt2_grad.py --reference` must name the same one.
- **SC vs. PC changes one scalar.** The Lagrangian is a statement about the reference, not about the contraction scheme. `--pc` replaces E_{corr}, PySCF's SC sum or the GPU's SC walk, by the PC energy (block2's offline, the GPU's PC walk on the card). Through E_{corr} it also replaces the two source terms. Everything else is unchanged: the constraint set, the Jacobian, the frozen path and the solve order. PC adds its own difference-quotient steps and a conditioning guard at every displaced CI vector (§10).

The Z-vector solve is offline by design. The repo's shape is "offline Python generates the reference, the C++ GPU demo reproduces it", and a PySCF solve inside the runtime would invert it. The GPU's job is the one expensive derivative of the NEVPT2 energy, its source terms. A gradient built from the GPU's dump *is* the NEVPT2 nuclear gradient, and is gated as one (`docs/testing.md`, "The GPU as the source-term provider").

stage	where	equations
the source terms Y_{xy}, y_I	GPU, reverse mode (`--dump-source-terms`)	§4.2
	or offline, central differences of PySCF/block2	§4.3
the constraint Jacobian, $\partial(Hc)/\partial\kappa$	offline, one κ sweep	§4.3
the Z-vector solve	offline only	§6, §7
the skeleton derivative and $\Lambda \cdot \partial g/\partial\xi$	offline, frozen-path differences of Φ	§5
the numerical gradient `grad_num`	offline, re-converged energies	§8

Table 2: Who computes what. Only the first row has a GPU path. Nothing in `src/` or `apps/` differentiates with respect to a nuclear coordinate (`docs/gradients.md`, §1).

2 Notation and conventions

Orbital spaces. i, j label core orbitals, t, u, v, w active orbitals and r, s virtual orbitals. In this document p, q, x, y label a *general* MO, any of the three; `docs/energy.tex` uses p, q for active ones. The counts are n_c, n_a, n_v, $n_{\mathrm{mo}} = n_c + n_a + n_v$. On the CASCI branch a second partition is in play, the RHF occupied space O and virtual space V. The active space straddles it: $\mathrm{act}_O = \mathrm{act} \cap O$ and $\mathrm{act}_V = \mathrm{act} \cap V$ (`docs/gradient-theory.md`, §0). I, J label determinants of the CI vector c. ξ is one Cartesian nuclear coordinate.

Orbital rotations. $C(\kappa) = C\, e^{\kappa}$ with κ real antisymmetric, so that

$$\delta h_{pq} = \sum_x \big(\kappa_{xp} h_{xq} + \kappa_{xq} h_{px}\big), \tag{1}$$

$$\delta(pq|rs) = \sum_x \big(\kappa_{xp}(xq|rs) + \kappa_{xq}(px|rs) + \kappa_{xr}(pq|xs) + \kappa_{xs}(pq|rx)\big), \tag{2}$$

in chemists' notation. The offline code steps a single pair (p, q), $p < q$, by the exact Givens rotation $C \exp\big(\theta(E_{pq} - E_{qp})\big)$ (`givens`).

Fock matrices. F is the **RHF** Fock matrix, built from the RHF density $2\,CP_OC^{\mathsf{T}}$ and so from O alone. f is the **generalised** Fock matrix, built from the zeroth-order density:

$$h^{\mathrm{eff}}_{pq} = h_{pq} + \sum_j \big[2(pq|jj) - (pj|jq)\big], \qquad f_{pq} = h^{\mathrm{eff}}_{pq} + \sum_{tu} \Gamma^{(1)}_{tu}\big[(pq|tu) - \tfrac{1}{2}(pt|uq)\big], \qquad \varepsilon_p = f_{pp}. \tag{3}$$

ε_i and ε_r are the golden's `e_core` and `e_virt`, and they move with the CI vector through $\Gamma^{(1)}$.

Density matrices. $\Gamma^{(1)}, \Gamma^{(2)}, \Gamma^{(3)}, F^{\mathrm{ca}}, F^{\mathrm{ac}}$ are those of `docs/energy.tex`, §2. $E_{tu} = \sum_\sigma a^\dagger_{t\sigma} a_{u\sigma}$. $P = 1 - |c\rangle\langle c|$ projects the state out of the determinant space.

Constants of the code. `GRAD_NUM_TOL` $= 10^{-6}$ (the gate against `grad_num`, and every ladder's tolerance); `PC_GAP_FLOOR` $= 10^{-6}\,E_h$ and `PC_ZERO_TOL` $= 10^{-8}$ (the degenerate-pair rule, §6); the difference-quotient steps of Table 3; and the ladder rule's window $(3.2, 4.8)$ and run length 3 (§13). All are in `nevpt2_grad.py` and `nevpt2_numgrad.py`. Why each step has its value is `docs/testing.md`, "The step constants, measured". Why the gate is not this repo's usual 10^{-7} is `docs/testing.md`, "The analytic gradient against the numerical one": both sides are finite differences.

step	differences	SC	PC
`KAPPA_STEP`	$E_{\rm corr}$, F, f, Hc along one κ pair	2^{-17}	10^{-4}
`CI_STEP`	$E_{\rm corr}$ along Pe_I	2^{-17}	10^{-7}
`NUC_STEP`	Φ along ξ, frozen path (bohr)	2^{-13}	0.0025
`STEP_BOHR`	re-converged E along ξ (`nevpt2_numgrad.py`)	0.0025	0.0025

Table 3: The difference-quotient steps. PC's CI step is not chosen for truncation error: it is the step at which the PC metric's lifted null modes stay below the cut (§10).

3 What is differentiated

The total energy is $E = E_{\rm CAS} + E_{\rm corr}$, a function of the geometry, the MO coefficients C and the CI vector c (Park Eq. 21). The amplitude dependence drops out because the Hylleraas functional is stationary in the amplitudes, for either reference. $E_{\rm CAS} = \langle c|H|c\rangle$ includes the nuclear repulsion and the core energy.

The SC finish, and its three multipliers. Each SC class reaches C and c only through three scalars per external tuple t: the norm N_t, the active part H_t of the Dyall expectation value, and the orbital-energy sum Δ_t (`docs/energy.tex`, §6). With the amplitude T_t,

$$\begin{gathered} T_t = -\frac{1}{\Delta_t + H_t/N_t}, \qquad E_t = T_t N_t, \\ P_t = \frac{\partial E}{\partial N_t} = 2T_t + \Delta_t T_t^2, \qquad Q_t = \frac{\partial E}{\partial H_t} = T_t^2, \qquad R_t = \frac{\partial E}{\partial \Delta_t} = N_t T_t^2, \end{gathered} \tag{4}$$

which are Park's Eqs. 36–38 and satisfy $P_t N_t + Q_t H_t = E_t$ (Eq. 40). This is the only hand-differentiated expression on the SC side; everything downstream is P, Q and R contracted back against integrals and densities (`nevpt2.sc_amplitudes`, `nevpt2.energy_finish`).

The PC finish. A PC class solves in the span of its internal functions, $E_t = -c_\kappa\, b_t^{\mathsf T} A_t^{+} b_t$ with $A_t = K + \Delta_t S$ and $b_t = Sx_t + S_1 h_t$ (`docs/energy.tex`, Eq. (29); c_κ the class's coupling factor). With $x = A_t^{+} b_t$, the solve the energy already ran,

$$\dot E_t = -c_\kappa \left(2\, x^{\mathsf T} \dot b_t - x^{\mathsf T} \dot A_t\, x\right), \tag{5}$$

so no eigenvalue or eigenvector derivative appears. This holds only where the kept rank of S is locally constant. The energy's gap refusal enforces that, and at an eigenstate K annihilates S's dropped space, which makes the two terms of Eq. (5) exact (`docs/gradients.md`, §7).

What carries the CI vector. The RDMs, and through $\Gamma^{(1)}$ in Eq. (3) the orbital energies. ε_p is affine in $\Gamma^{(1)}$:

$$\varepsilon_p = h^{\rm eff}_{pp} + \sum_{tu} \Gamma^{(1)}_{tu}\, W_p[t,u], \qquad W_p[t,u] = (pp|tu) - \tfrac{1}{2}(pt|up), \qquad p \notin \text{act}. \tag{6}$$

Holding the orbital energies fixed differentiates a different functional, and that omission is invisible to any check whose two sides share it (`docs/gradients.md`, §5.5).

4 Shared stage A: the source terms Y and y

4.1 What the Lagrangian reads

Park's Eqs. 26–27 define the two source terms as derivatives of $E_{\rm corr}$ at *fixed* constraints, with no solve involved:

$$Y_{xy} = \frac{\partial E_{\rm corr}}{\partial \kappa_{xy}}, \qquad y_I = \frac{\partial E_{\rm corr}}{\partial c_I}. \tag{7}$$

κ is antisymmetric and c is normalised, so the solve reads only two combinations:

$$A_{pq} = Y_{pq} - Y_{qp}, \qquad Py = y - (c \cdot y)\, c. \tag{8}$$

Every block of A has a reader. Core–core and virtual–virtual feed (O2)'s multipliers, active–active feeds the gauge identity, and the three mixed blocks feed the orbital Z-vector equation (`docs/gradient-theory.md`, §2.1). The source terms are the same on both branches: they differentiate the NEVPT2 energy, which does not know how its orbitals were made. There are two providers.

4.2 Provider 1: the GPU, by reverse mode

The CI source term. `scCiSourceDevice` runs the class energies' own slab walk once more. Each class is a list of norm and H terms in the same subscript strings as its energy einsums. One routine evaluates the list and differentiates it by rewriting the subscripts per operand, so no derivative is hand-transcribed. The walk leaves an accumulator $M_G = \partial E/\partial G$ per metric and Dyall block G. Every block is linear in the RDMs, so a reverse-topological sweep through the blocks' own builders gives five coefficient tensors:

$$\bar\rho = \frac{\partial E}{\partial \rho} = \sum_G \Big(\frac{\partial G}{\partial \rho}\Big)^{\mathsf T} M_G, \qquad \rho \in \{\Gamma^{(1)}, \Gamma^{(2)}, \Gamma^{(3)}, F^{\rm ca}, F^{\rm ac}\}. \tag{9}$$

The orbital energies add one more $\Gamma^{(1)}$ coefficient. Each class scatters its R_t onto the ε_p its Δ_t reads, and the sum enters through Eq. (6):

$$\bar\Gamma^{(1)}_{tu} \mathrel{+}= \sum_{p \notin \rm act} \bar\varepsilon_p\, W_p[t,u], \qquad \bar\varepsilon_p = \sum_t R_t\, \frac{\partial \Delta_t}{\partial \varepsilon_p}. \tag{10}$$

Then

$$y_I = \sum_\rho \Big\langle \bar\rho,\ \frac{\partial \rho}{\partial c_I} \Big\rangle, \tag{11}$$

taken without ever forming $\partial\Gamma^{(3)}/\partial c$ (an $n_a^6\, n_{\rm det}$ object) or the 4-RDM. One 3-body sigma vector carries $\bar\Gamma^{(1)}$, $\bar\Gamma^{(2)}$ and $\bar\Gamma^{(3)}$, because the trace of a one-body generator is the electron count on any determinant. A forward and a dagger 4-body contracted sigma for each F order carry the digests, through one $(n_a^2, n_{\rm det})$ intermediate (`docs/gradients.md`, §5.3–5.5; `nevpt2.sigma_vector`).

The orbital source term. Along a rotation at fixed c, every stored array with an index the rotation touches moves. The stored two-electron blocks move by Eq. (2), slot by slot. The one-electron blocks move, their core dressing promoted with them. The orbital energies move. And the digests move, because they are the active integrals contracted with the 4-RDM. The pairing is

$$\langle \kappa, Y\rangle = \underbrace{\langle \bar I,\ \dot I(\kappa)\rangle + \sum_t R_t\, \dot\Delta_t(\kappa)}_{\text{frozen digests}} + \underbrace{\langle \bar v,\ \dot v(\kappa)\rangle}_{\text{the digests' term}}, \tag{12}$$

where I is every stored block, $\bar{I}$ its pseudodensity (Park Eq. 41 and the Eqs. 42–47 intermediates), and $\bar{v} = \partial E/\partial v$ through $F^{\mathrm{ca}}, F^{\mathrm{ac}}$ (with v the active block, `docs/energy.tex`, Eq. (1)). $\dot{I}(\kappa)$ is the *promotion rule*: replace one slot of a block by a general index and contract it with κ. Each right-hand side is one of the gradient sidecar's one-general-index arrays (Appendix A). $\bar{v}$ comes from running the digest build's $f3$ path backwards, the *transpose* (`buildDigestAdjointDevice`). The plain build's `produce` kernel regenerates L tile by tile, so the 4-RDM is never formed (`docs/gradients.md`, §6). The active–active part of the directly contractible terms is symmetric, so it is invisible to an antisymmetric κ, and the matrix Y is assembled by probing Eq. (12) along basis directions (`scFullRotationMatrix`).

PC. The same walk with each PC class's solve differentiated through its own S, K and S_1, Eq. (5) read as an adjoint:

$$\frac{\partial E}{\partial K} = U\,\mathrm{sym}(m)\,U^{\mathsf{T}}, \quad \frac{\partial E}{\partial S} = \mathrm{sym}(U\,\bar{T}^{\mathsf{T}}) + U\,\mathrm{sym}\Big(\textstyle\sum_t \Delta_t m_t\Big)U^{\mathsf{T}}, \quad \frac{\partial E}{\partial S_1} = U\,\bar{t}_1^{\mathsf{T}}, \quad R_t = \frac{\partial E}{\partial \Delta_t}, \tag{13}$$

with U, T, t_1 the solve's eigenbasis quantities (`docs/energy.tex`, Eqs. (30)–(33)), m_t the tuple's weight in that basis, and $\bar{T}$, $\bar{t}_1$ the slab einsum's adjoint in its basis operand. K is SC's own Dyall blocks and S, S_1 are permutations of SC's metrics, so Eq. (9)'s sweep carries them down unchanged. A class that the energy's gap or denominator test refuses carries that refusal out of every entry point; it never contributes a zero (`docs/gradients.md`, §5.6, §6.1).

The gates the GPU holds itself to. Both source terms are also computed in forward mode, a tangent $\dot{E}(u)$ along a CI direction and $\dot{E}(\kappa)$ along a rotation. The two modes share no code past the slab walk, so their agreement is an exact identity with no step size in it:

$$\langle u, y\rangle = \dot{E}(u), \qquad \langle \kappa, Y\rangle = \dot{E}_{\mathrm{frozen}}(\kappa) + \dot{E}_{\mathrm{digest}}(\kappa). \tag{14}$$

The dualities cannot see a term both modes omit. The orbital energies' response and the digests' term are therefore also held to PySCF's exact derivative and to finite differences of the unmodified energy (`docs/gradients.md`, §2 and §5.5).

Algorithm 1 `nevpt2_sc_pseudodensity --dump-source-terms` *P* (`apps/sc_pseudodensity/source_dump.cppm`)

Input: golden g and its `.gradgold` sidecar; an exact digest (`--cublas`, `--ozaki`, `--fused-digest` refused); `--pc` optional

1: $(\Gamma^{(3)}, F^{\mathrm{ca}}, F^{\mathrm{ac}}) \leftarrow$ `buildRdmsDevice` ▷ `docs/energy.tex`, Alg. 2
2: $W \leftarrow$ `orbitalEnergyResponse`(sidecar) ▷ Eq. (6)
3: $\bar{v} \leftarrow$ the digests' transpose ▷ `scDigestAdjointDevice`, or PC's
4: $Y \leftarrow$ probe Eq. (12) along every MO pair ▷ `scFullRotationMatrix`
5: $y \leftarrow$ Eqs. (9)–(11) with W ▷ `scCiSourceDevice`, or `pcCiSourceDevice`
6: along six seeded directions κ_d: $\dot{E}_{\mathrm{frozen}}(\kappa_d)$, $\dot{E}_{\mathrm{digest}}(\kappa_d)$
7: **gate:** the matrix is the pairing; $\dot{E}(\kappa_d) = \langle \kappa_d, Y\rangle$ to 10^{-7}; y's walk reproduces E_{corr} to 10^{-7}; PC: the integral halves' duality too
8: write P (`.gradsrc`, an `.npz`): `kappa_source`, `ci_source`, `ci`, n_c, n_a, n_v, `e_corr`, `pc`, `edot_*`; read it back bit for bit

Carrying the dump into the gradient's basis. The GPU knows one active basis, the golden's natural orbitals. On the CASCI branch `nevpt2_grad.py` holds the state in the eigenbasis of F's active block (§6), reached by an active rotation W_a that carries c along. `load_source_terms` therefore first checks that the dump's `ci` is the golden's up to a sign σ (to `DUMP_CI_TOL`), that its partition is the state's, that its `e_corr` matches, and that its scheme (SC or PC) is the run's. It then transforms

$$Y' = W^{\mathsf{T}} Y\, W, \quad W = 1 \oplus W_a, \qquad y' = \mathcal{T}(W_a)\,(\sigma y), \tag{15}$$

with $\mathcal{T}(W_a)$ the CI transform for the orbital rotation (orthogonal, so y transforms as c does). E is even in c, so y is odd, and the sign goes on y. On the CASSCF branch the state is the golden's own basis and $W_a = 1$.

4.3 Provider 2: offline central differences

`--source pyscf`, the default, differentiates PySCF's SC energy, or block2's PC energy with `--pc`, and needs no card. One pass over every non-gauge pair $k = (p, q)$ (`kappa_sweep`) produces three objects at once:

$$A_{pq} = \frac{E(C\,G_{pq}(h)) - E(C\,G_{pq}(-h))}{2h}, \tag{16}$$

$$M_{km} = \frac{\partial G_m}{\partial \kappa_k}, \qquad Q_{kI} = \frac{\partial (Hc)_I}{\partial \kappa_k}, \tag{17}$$

at $h =$ `KAPPA_STEP` and fixed c, with G_{pq} the Givens rotation and G_m the orbital constraints of §6–7. M is the constraint Jacobian. Q makes the CI relaxation Qz available for any z found later. M is differenced rather than written out because f is not covariant under a rotation that mixes the core and active index ranges. The CI source term is differenced along directions *orthogonal to the state* and inside the state's spatial irrep (`allowed_determinants`):

$$y_I = \|Pe_I\|\, \frac{E_{\mathrm{corr}}(c + h\,d_I) - E_{\mathrm{corr}}(c - h\,d_I)}{2h}, \qquad d_I = \frac{Pe_I}{\|Pe_I\|}, \tag{18}$$

at $h =$ `CI_STEP`, with ε rebuilt from Eq. (3) at every displaced c. Both restrictions are load-bearing. Off the unit sphere the hole-side metrics' constants are no longer hole RDMs and the SC finish has poles. Outside the irrep a symmetry-empty perturber fills linearly in h, and the limit, though it converges, is not linear in the direction (`nevpt2_grad.py`, `pyscf_ci_source`). When a dump supplies Y, the sweep still runs to build M and Q, but with E_{corr} dropped, and the dump's A is added to the sweep's E_{CAS} half.

`--source both` solves twice on one sweep, once from the dump and once from PySCF's or block2's quotients. It compares the two source terms block by block and prints both gradients beside `grad_num`, so a difference between them can only come from the source terms.

5 Shared stage B: the frozen orthonormal path, and Φ

Given the multipliers Λ, the gradient is the nuclear derivative of one scalar along a path on which the wavefunction parameters do not respond:

$$C(\xi) = S(\xi)^{-1/2}\, S(\xi_0)^{1/2}\, C(\xi_0), \qquad c(\xi) = c(\xi_0), \tag{19}$$

with S the AO overlap (`frozen_coeff`). Then $C(\xi)^{\mathsf{T}} S(\xi) C(\xi) = 1$ at every geometry. The orthonormality constraint holds identically along the path and needs no multiplier, so no energy-weighted density term appears. With

$$\Phi(\xi) = E_{\mathrm{CAS}}(\xi) + E_{\mathrm{corr}}(\xi) + \sum_m \lambda_m G_m(\xi) + \Lambda_{\mathrm{CI}}(\xi), \tag{20}$$

where Λ_{CI} is the CI constraint's term ($\langle z|H|c\rangle$ on the CASCI branch, $z \cdot g_{\mathrm{ci}}$ on the CASSCF one), every quantity is evaluated at $(C(\xi), c)$ on the displaced molecule's own integrals. The multipliers satisfy their equations, so

$$\frac{dE}{d\xi} = \frac{d\Phi}{d\xi} \approx \frac{4\,g(h/2) - g(h)}{3}, \qquad g(h) = \frac{\Phi(\xi + h) - \Phi(\xi - h)}{2h}, \tag{21}$$

at $h =$ `NUC_STEP`. The Richardson bar $|R - g(h/2)|$ is printed with the gradient. The two halves are reported separately: `get_ham_response` is the derivative of $E_{\mathrm{CAS}} + E_{\mathrm{corr}}$, and `get_LdotJnuc` is the derivative of the constraint terms.

This is a Lagrangian gradient whose skeleton derivatives are differenced, not contracted against derivative integrals, and that is a stated limitation. All the analytic content is exercised: the constraint set, the Z-vector equations and the stationarity that makes a frozen path sufficient. Replacing the differences with PySCF's derivative integrals would need the NEVPT2 relaxed densities, and no flag writes those out (§14). Nothing re-solves along the path, so every Cartesian component is differenced, including the ones that are zero by symmetry. They come out as the zeros they are, which is a free check (§8).

Algorithm 2 `Gradients.kernel`: the frozen-path derivative

Input: the state (C_0, c) at ξ_0; multipliers λ, z; step h

$S_0^{1/2} \leftarrow$ `sym_power`$(S(\xi_0), \frac{1}{2})$
for each atom and Cartesian component ξ **do** ▷ all $3N$, zero-by-symmetry included
  for $s \in \{h, h/2\}$ **do**
    for $\pm$ **do**
      build the molecule at $\xi \pm s$; its RHF object, integrals and, fitted, its own `with_df`
      $C \leftarrow S(\xi \pm s)^{-1/2} S_0^{1/2} C_0$ ▷ Eq. (19)
      $\Phi_\pm \leftarrow E_{\mathrm{CAS}} + E_{\mathrm{corr}} + \lambda \cdot G + \Lambda_{\mathrm{CI}}$ at (C, c) ▷ Eq. (20)
    $g(s) \leftarrow (\Phi_+ - \Phi_-)/2s$
  $\partial E/\partial\xi \leftarrow [4g(h/2) - g(h)]/3$; bar $\leftarrow |\cdot - g(h/2)|$

6 The CASCI-branch Lagrangian and its solve

6.1 The constraints

On RHF orbitals, canonicalised with `cas_natorb=True`, core, active and virtual are each spanned by a fixed set of RHF canonical orbitals (`docs/gradient-theory.md`, §1.2). The constraint set is

$$\text{(O1)}: \; F_{pq} = 0 \text{ for } p, q \text{ in different spaces of } \{\mathrm{core}, \mathrm{act}_O, \mathrm{act}_V, \mathrm{virt}\}, \tag{22}$$

$$\text{(O2)}: \; f_{ij} = 0 \; (i \neq j \text{ core}), \quad f_{rs} = 0 \; (r \neq s \text{ virtual}), \tag{23}$$

$$\text{(C1)}: \; (H - E_{\mathrm{CAS}})\, c = 0, \quad c^{\mathsf{T}} c = 1, \tag{24}$$

and the Lagrangian is

$$L = E_{\mathrm{CAS}} + E_{\mathrm{corr}} + \sum_{m\in(\mathrm{O1})} \lambda_m F_{p_m q_m} + \sum_{m\in(\mathrm{O2})} \lambda_m f_{p_m q_m} + \langle z|H - E_{\mathrm{CAS}}|c\rangle. \tag{25}$$

(O1)'s blocks between O and V are RHF Brillouin conditions. Its blocks within O (core–act$_O$) and within V (act$_V$–virtual) hold by the active-space *selection* (§14).

The gauges are act$_O$-internal and act$_V$-internal rotations, and no others. An act$_O$–act$_V$ rotation leaves the energy alone, being an active–active rotation of a full CI, but it moves the RHF occupied space and therefore F. So $F_{\mathrm{act}_O,\mathrm{act}_V} = 0$ is a genuine Brillouin condition with a multiplier of its own. For that reason the state is held in the basis that diagonalises F's active block, with the CI vector transformed to match, so that act$_O$ and act$_V$ are index ranges (`build_state`; `docs/gradient-theory.md`, §2.6). The rebuild is checked: E_{corr} must not move under the rotation, and F's active block must come out diagonal. `kappa_pairs` is every $p < q$ except those two gauge blocks. `constraints` pairs them one to one with (O1) (between spaces) and (O2) (inside core or inside virtual), and asserts the count, so stationarity is a *square* system.

6.2 The stationarity equations

Differentiating Eq. (25):

$$\frac{\partial L}{\partial c}: \quad P(H - E_{\mathrm{CAS}})P\, z = -P(y + y^c), \tag{26}$$

$$\frac{\partial L}{\partial \kappa_k}: \quad A_k + (Qz)_k + \sum_m M_{km}\lambda_m = 0 \quad \text{for every non-gauge pair } k, \tag{27}$$

with A, M, Q from Eqs. (16)–(17) (or A from the dump) and y masked to the state's irrep. F is built from the RHF density and has no CI dependence, so Eq. (26) contains no (O1) term. That is the decoupling of `docs/gradient-theory.md`, §2.2. One coupling survives. f is built from $\Gamma^{(1)}$ and so does depend on c, which puts (O2)'s multipliers into the CI right-hand side as a one-body sigma vector (`ci_coupling`):

$$y^c_I = \sum_{tu} \left(\mathcal{W} + \mathcal{W}^{\mathsf{T}}\right)_{tu} (E_{tu}c)_I, \qquad \mathcal{W}_{tu} = \sum_{m\in(\mathrm{O2})} \lambda_m \left[(p_m q_m|tu) - \tfrac{1}{2}(p_m t|u q_m)\right]. \tag{28}$$

The solve. Eq. (26) goes to preconditioned conjugate gradient: $P(H - E_{\mathrm{CAS}})P$ is positive semidefinite on the projected space. The preconditioner projects, and the answer is projected again, $z \leftarrow z - (c\cdot z)c$. $|c\rangle$ is an exact null vector, so a Jacobi preconditioner would otherwise let z drift along it at no cost to the residual. Φ drops $-E'_{\mathrm{CAS}}(\xi)\,(z\cdot c)$ on the assumption that $z\cdot c = 0$. The orbital system, Eq. (27), is square and indefinite by construction: the (O2) diagonal $\varepsilon_q - \varepsilon_p$ changes sign with the pair order. It gets a direct least-squares solve (`JACOBIAN_RCOND`), with GMRES through the same operator beside it as an independent residual. The two equations are iterated to a fixed point, because y^c reads λ and the orbital right-hand side reads Qz. The iteration count is printed.

Degenerate (O2) pairs. Where $|\varepsilon_p - \varepsilon_q| <$ `PC_GAP_FLOOR`, the pair's multiplier is a division the state does not determine. If $|A_{pq}| \le$ `PC_ZERO_TOL`, the pair and its constraint are dropped together, which keeps the system square, and the multiplier is left at zero. An abelian subgroup puts two degenerate partners in different irreps, so the source term vanishes there too and zero is the right representative. Otherwise the run is refused rather than divided through (`degenerate_constraints`).

The theory's triangular order falls out. On (O2)'s rows M is diagonal in the pseudocanonical gauge. Those rows reduce to Park's Eqs. 56–57 (Appendix B), the selection-boundary rows to two small Sylvester equations, and the rest to CPHF on the O–V block (`docs/gradient-theory.md`, §2.3–2.5). The code imposes none of that. It solves the one square system, so a response block that the triangular form assumes to vanish is never relied on.

Algorithm 3 `Gradients` (CASCI branch): `prepare`, `solve_multipliers`, `kernel`

Input: the rebuilt state; source terms (A, y) from a dump or `None`

rotate the active block to F's eigenbasis, CI vector along; check E_{corr} unmoved and (O1), (O2) ▷ §6
$(A_{\mathrm{sweep}}, M, Q) \leftarrow$ `kappa_sweep` ▷ Eqs. (16)–(17); E_{corr} dropped under a dump
$A \leftarrow A_{\mathrm{sweep}}$, or $A^{E_{\mathrm{CAS}}}_{\mathrm{sweep}} + A_{\mathrm{dump}}$; $y \leftarrow$ Eq. (18) or the dump's
$\lambda \leftarrow 0$; $z \leftarrow 0$
repeat ▷ the CI/(O2) fixed point
    $b_{\mathrm{ci}} \leftarrow P\big(y + y^c(\lambda)\big)$ ▷ Eq. (28)
    $z \leftarrow \textsc{Pcg}\big(P(H - E_{\mathrm{CAS}})P,\ -b_{\mathrm{ci}}\big)$; $z \leftarrow Pz$
    drop degenerate (O2) pairs, or refuse
    $\lambda \leftarrow \textsc{Lstsq}\big(M,\ -(A + Qz)\big)$; GMRES beside it
until the step in (z, λ) is below 10^{-12}
report the gauge identity: $\max|A + Qz|$ over the act_O-, act_V-internal rotations
$\partial E/\partial\xi \leftarrow$ Algorithm 2 with $\lambda \cdot G = \lambda \cdot (F, f)$ and $\Lambda_{\mathrm{CI}} = z \cdot (H(\xi)c)$

7 The CASSCF-branch Lagrangian and its solve

On converged CASSCF orbitals, Park's Eq. 23 holds as written (`docs/gradient-theory.md`, §1.4). Its first line, CASSCF stationarity, replaces (O1):

$$L = E_{\mathrm{CAS}} + E_{\mathrm{corr}} + Z \cdot g_{\mathrm{orb}}(\kappa, c) + z \cdot g_{\mathrm{ci}}(\kappa, c) + \sum_{m \in (\mathrm{O2})} z^c_m \, f_{p_m q_m}, \tag{29}$$

with $g = (g_{\mathrm{orb}}, g_{\mathrm{ci}})$ PySCF's second-order CASSCF gradient, `newton_casscf.gen_g_hop`'s `g_all`. The orbital part g_{orb} runs over the non-redundant core–active, core–virtual and active–virtual rotations (constraint kind (S1)), and $g_{\mathrm{ci}} = 2(H - E_{\mathrm{CAS}})c$ is (C1). There is no F, no (O1), no $\mathrm{act}_O/\mathrm{act}_V$ split and no rotation of the state. Every active–active rotation is a gauge.

The solve, in two steps and one pass. First z^c, alone, from (O2)'s own rows. At a stationary point no other constraint responds to a core–core or virtual–virtual rotation (`solve_o2`, with the same degenerate-pair rule as §6):

$$M_{oo}\, z^c = -A_{oo}. \tag{30}$$

Then one coupled system (`solve_coupled`), with $J = \partial g/\partial x$ (`gen_g_hop`'s `h_op`, the CASSCF orbital+CI Hessian in its own step variables x), solved on a basis B: the identity on the orbital variables, and an orthonormal basis of the in-sector determinants with c projected out:

$$\begin{gathered} \big(B^{\mathsf{T}} J B\big)^{\mathsf{T}}\, \ell = -B^{\mathsf{T}} b, \qquad \begin{pmatrix} Z \\ z \end{pmatrix} = B\ell, \\ b = \begin{pmatrix} -\big(A_{pq} + \sum_{o \in (\mathrm{O2})} M_{ko}\, z^c_o\big)_{k=(p,q)} \\ P\big(y + y^c(z^c)\big) \end{pmatrix}. \end{gathered} \tag{31}$$

The minus sign on the orbital rows is the Givens convention: the rotation by θ on (p, q) is `gen_g_hop`'s step $x_{(q,p)} = -\theta$. By default $B^{\mathsf{T}}JB$ is built densely, one `h_op` product per column, and LU-factorised. The solve is exact, with no preconditioner, level shift or threshold. Its symmetric part is also diagonalised, and the modes nearest singular are printed with their weight on the right-hand side, so a near-null or negative mode is seen directly rather than inferred. `--cp-solve cg` runs the base class's conjugate gradient through the same operator, for a case too large to build densely.

Algorithm 4 `CasscfGradients` (CASSCF branch)

Input: the state on `--mo-from`'s orbitals, unrotated; source terms as Algorithm 3

1: $(A, M_{oo}) \leftarrow$ `kappa_sweep` ▷ only (O2)'s columns of M are differenced
2: $(g, J) \leftarrow$ `gen_g_hop`(C, c); report $\|g_{\mathrm{orb}}\|$, $\|g_{\mathrm{ci}}\|$ (the residual stationarity)
3: $z^c \leftarrow$ Eq. (30) ▷ degenerate pairs left at zero, or refused
4: $B \leftarrow$ orbital variables $\oplus$ (in-sector determinants $\perp c$)
5: $J_{\mathrm{red}} \leftarrow B^{\mathsf{T}}JB$, column by column; $\ell \leftarrow J_{\mathrm{red}}^{-\mathsf{T}}(-B^{\mathsf{T}}b)$ ▷ Eq. (31)
6: report $\|J_{\mathrm{red}}^{\mathsf{T}}\ell + B^{\mathsf{T}}b\|$, the same ℓ through `h_op`, J's asymmetry and its spectrum
7: $(Z, z) \leftarrow B\ell$
8: $\partial E/\partial\xi \leftarrow$ Algorithm 2 with $\lambda \cdot G = Z \cdot g_{\mathrm{orb}} + z^c \cdot f$ and $\Lambda_{\mathrm{CI}} = z \cdot g_{\mathrm{ci}}$, g from `gen_g_hop` at each displaced geometry

8 The numerical gradient, and the gate

`nevpt2_numgrad.py` differences re-converged energies built by `generate_golden._build_state` itself, at the same settings as the golden. Those settings are single-threading, a 10^{-12} Davidson tolerance, the point group detected, and the same irrep-selected active space. For each probed component, with $h =$ `STEP_BOHR`,

$$g(s) = \frac{E(\xi + s) - E(\xi - s)}{2s}, \qquad R(s) = \frac{4\,g(s/2) - g(s)}{3}, \tag{32}$$
$$\texttt{grad_num} = R(h/2), \qquad \texttt{grad_num_residual} = |R(h/2) - R(h)|,$$

so each component costs six displaced energies. The residual is an error bar on the stored number.

In-plane displacements only, and the refusal. The active space is selected by irrep, and irrep labels exist only while the point group does. A displacement whose detected point group differs from the reference's is refused (`_check_point_group`). So is one whose core or active irrep labels differ at an unchanged group (`_check_selection`). The components such a displacement would probe are zero by symmetry at a planar geometry. `--components auto` records them as exact zeros and leaves them to the analytic side as a free check (`docs/gradient-theory.md`, §4.3).

On a CASSCF reference (`--reference casscf`) every displaced state is re-optimised from a cold start through the generator's own optimiser, then finished to a tighter orbital gradient, then given the golden's `--mo-from` CASCI and NEVPT2. A displaced optimisation that lands on another stationary point would give a number from a different reference, so every displaced state passes a continuity guard before it is differenced. The guard refuses an energy off its first-order prediction, an active subspace or CI vector that moved too far, and an unconverged optimiser. A warm start on a few geometries probes for a second solution (`docs/reference-data.md`, "The numerical gradient of a CASSCF reference"). Its PC twin runs no sweep: `--pc-states` differences the GPU's PC energies on the finished sweep's kept states (`pc_on_states.py`).

The gate (`nevpt2_grad.py`'s last step) compares each probed component α with `grad_num`:

$$\max_{\alpha} |g_\alpha - \texttt{grad_num}_\alpha| < \texttt{GRAD_NUM_TOL} = 10^{-6}, \tag{33}$$

and prints `PASS:` or `FAIL:`, exiting 1 on a failure. The source is the sidecar's `grad_num`, or an `.npz` named with `--grad-num`; the file must be of the same case, reference and functional (`grad_num`, `grad_num_pc`, or `grad_casci` under `--casci-only`). Two more checks are free and always reported: the gradient summed over atoms (translational invariance), and $|g|$ on the axes along which every atom shares a coordinate, i.e. zero by symmetry (`zero_axes`). **There is no ctest entry for the gate.** It would put PySCF and block2 in the test path, and the gate needs no card. Only the GPU dump it reads is a ctest entry (`docs/testing.md`, "Why there is no ctest entry for it").

9 Path 1: SC-NEVPT2 on a CASCI reference

Offline, before. The golden is the energy's (`docs/energy.tex`, Path 1, conventional integrals here). Beside it, `generate_golden.py --gradgold` writes the `.gradgold` sidecar: the one-general-index integral inventory of Appendix A, the full-range one-electron and Fock matrices, F and `mo_coeff`. `nevpt2_numgrad.py` appends `grad_num` to it, or writes an `.npz` for a case whose committed sidecar carries none.

Algorithm 5 Path 1, SC on CASCI orbitals

Offline (`generate_golden.py --gradgold`; `nevpt2_numgrad.py [--npz]`)
1: golden and sidecar, as Path 1 of `docs/energy.tex` plus the inventory
2: `grad_num` $\leftarrow$ Eq. (32) over the in-plane components
GPU (`nevpt2_sc_pseudodensity --golden` *g* `--tiles 3 --dump-source-terms` *P*)
3: $(Y, y) \leftarrow$ Algorithm 1; **PASS** on its own gates
Offline (`nevpt2_grad.py --case` *c* `--source dump|both --source-path` *P*)
4: rebuild the golden's state; refuse it unless E_{CAS}, E_{corr} are the golden's to `REF_STATE_TOL`
5: carry (Y, y) into the F-eigenbasis ▷ Eq. (15)
6: $(\lambda, z) \leftarrow$ Algorithm 3; $g \leftarrow$ Algorithm 2
7: the residual-independent checks ▷ §13
8: **PASS** iff Eq. (33) holds and every residual-independent check passes

The equations are Eqs. (4)–(15), (19)–(21), (25)–(28) and (32)–(33). Without `--source`, Algorithm 5's GPU step is replaced by Eqs. (16) and (18) and needs no card. The cases are LiF CAS(4,6) and acrolein CAS(6,5), the planar C_s gradient-validation pair (acrolein's sidecar is gitignored), and N2 CAS(4,4). N2 is the deliberately symmetric case: six exactly degenerate inactive pairs, and a linear molecule that keeps its point group under every displacement. Its committed sidecar carries no `grad_num`, so the route there is `--npz`. The GPU dump's ctest entries are `nevpt2_sc_pseudodensity_dump_{n2_ccpvdz_cas44, n2_ccpvdz_cas88, lif_def2svp_cas46}`, plus the CAS(4,4) and LiF `_casscf` twins. The verdicts are `docs/testing.md`, "The analytic gradient against the numerical one" and "The GPU as the source-term provider".

9.1 The density-fitted variant

`--df` differentiates the density-fitted SC energy on the `_df` golden's state: a fitted RHF and a `DFCASCI` sharing its `with_df`, with `dfnevpt2._ERIS` called directly. Nothing else changes. The fitted RHF is

still an RHF, so (O1), (O2), (C1), the frozen path and the solve are this section's. **The fit's own nuclear derivative is in the quotient with no term written out**: each displaced geometry in Algorithm 2 builds its own fitting object, so the auxiliary functions and the metric $(P|Q)$ move with the nuclei. An orbital rotation, by contrast, never reaches the metric (`docs/gradient-theory.md`, §3.8):

$$(pq|rs) = \sum_L B^L_{pq} B^L_{rs}, \qquad \delta B^L_{pq} = \sum_x (\kappa_{xp} B^L_{xq} + \kappa_{xq} B^L_{px}), \tag{34}$$

so the promotion rule acts on one B factor at a time, and every inventory array is a strip of B against a block of B. The GPU's fitted dump (`nevpt2_sc_pseudodensity --dump-source-terms` on the `_df` golden beside its `NEVPT2F1` sidecar) contracts that inventory out of the whole B and carries both terms of Y. The gate is the `NEVPT2F1` sidecar's `grad_num`, which `nevpt2_numgrad.py --df` wrote. The variant is SC, CASCI branch and N2 CAS(4,4) only. `--pc`, `--reference casscf`, `--casci-only` and `--grad-num` are refused with it (`docs/testing.md`, "The density-fitted gradient against the numerical one").

10 Path 2: PC-NEVPT2 on a CASCI reference

What `--pc` changes. E_{corr} becomes block2's PC energy (`generate_golden._block2_icmr`) in the quotients and in Φ. The GPU's dump becomes the PC one (`--pc --dump-source-terms`, marked `pc`; a dump of the other scheme is refused). Four things come with it:

- **The orbital energies, rebuilt at every point.** block2 runs `canonicalized = True` and uses the `mo_energy` it is handed, so every evaluation hands it Eq. (3)'s diagonal at that (C, c).
- **The 4-RDM, rebuilt when c moves.** It is cached across the κ sweep, keyed on the CI vector's bytes.
- **A PC answer must exist.** `_check_pc_conditioning` refuses LiF's state as a continuum through the cut, so `--pc` on LiF stops at the reference state. There is no PC gradient check on LiF, by design (`docs/testing.md`, "LiF has no PC gradient, and that is the answer").
- **Its own CI step, and the guard at every displaced CI vector.** A determinant-axis displacement is not a spin eigenfunction. It lifts the PC metric's structurally null modes, and a lifted mode past the cut is kept and divided by. That is an error of order h/h in the quotient, which no step ladder converges away and no residual sees. `CI_STEP_PC` keeps the lifted modes below the cut. Every PC energy at a CI vector other than the state's own runs the conditioning guard, and a rank test against the reference state's smallest kept singular value (`PC_RANK_KEPT_FRACTION`) catches a mode kept outright. A ledger counts evaluations and guard runs in two different places, so a path that skips the guard fails the run (`check_pc_guard_ledger`).

The equations are Path 1's, with Eqs. (5) and (13) in place of Eq. (4) on the GPU side. The cases are acrolein CAS(6,5) and N2 CAS(4,4). The cost is the κ sweep, one block2 PC solve per non-gauge rotation (`--workers N` forks it, bit-identically to the serial loop). The verdicts are `docs/testing.md`, "The analytic gradient against the numerical one", and `docs/pc-nevpt2.md`, "The source terms by reverse mode".

Algorithm 6 Path 2, PC on CASCI orbitals

Offline: Algorithm 5's lines 1–2, `nevpt2_numgrad.py --pc --npz N` (no sidecar is written)
GPU (`nevpt2_sc_pseudodensity ... --pc --dump-source-terms P`): Algorithm 1 with PC's walk
Offline (`nevpt2_grad.py --case c --pc [--source dump|both --source-path P] --grad-num N`)

1: Algorithm 5's offline lines, with block2's E_{corr}, the PC steps of Table 3, and the guard at every displaced c
2: **PASS** iff Eq. (33) holds against `grad_num_pc`, every residual-independent check passes, and the guard ledger balances

11 Path 3: SC-NEVPT2 on a CASSCF reference

Offline, before. The golden is the energy's Path 3: a CASCI on the committed `_casscf.mogold` orbitals, which pass the `--mo-from` stationarity gate (`docs/energy.tex`, Eq. (39)). The CASSCF-reference sidecar is written by `--mo-from --gradgold`. The numerical gradient is `nevpt2_numgrad.py --reference casscf`, which re-optimises every displaced state (§8).

Algorithm 7 Path 3, SC on CASSCF orbitals

Offline (`generate_golden.py --mo-from m --gradgold`; `nevpt2_numgrad.py --reference casscf --npz N`)

1: golden and sidecar on the CASSCF orbitals
2: `grad_num` ← Eq. (32) over re-optimised, continuity-guarded displaced states
GPU: Algorithm 1 on the `_casscf` golden, **unchanged**
Offline (`nevpt2_grad.py --case c --reference casscf [--source both --source-path P] [--grad-num N]`)
3: rebuild the state through `--mo-from`'s refusals (fingerprint, symmetry purity, the stationarity gate); assert (O2)
4: read the dump unrotated ▷ the state is the golden's basis
5: (z^c, Z, z) ← Algorithm 4; g ← Algorithm 2
6: the residual-independent checks, plus the Hessian-vector ladder ▷ §13
7: **PASS** iff Eq. (33) holds (where a `grad_num` exists) and every check passes

The equations are Eqs. (4)–(21), (29)–(31) and (32)–(33). The GPU side is reference-agnostic: source terms are derivatives at fixed orbitals, and the dump takes a `casscf` golden behind the source-term gate. The cases are N2 CAS(4,4) (the inactive-degeneracy probe), LiF CAS(4,6), and HSOH CAS(6,6), the C_1 case. HSOH keeps its point group under every displacement, so all $3N$ components are probed and none is zero by symmetry. Its committed `_casscf.gradgold` carries the re-optimised sweep's `grad_num`, which is read by default. Without a `grad_num` the gate is the residual-independent checks. The verdicts are `docs/testing.md`, "SC on a CASSCF reference: LiF, and the N2 degeneracy probe" and "SC on a CASSCF reference: the C1 case".

12 Path 4: PC-NEVPT2 on a CASSCF reference

What it is. Path 2's `--pc` fed into Path 3's coupled solve. Y, y and Φ's E_{corr} are the PC energy's, at the PC steps and under the guard. J, (S1), (C1), (O2) and B are the reference's and do not change.

Algorithm 8 Path 4, PC on CASSCF orbitals

Offline: Algorithm 7's sweep; then `pc_on_states.py` (the GPU's PC energy on each kept state) and `nevpt2_numgrad.py --reference casscf --pc-states S --npz N`
GPU: Algorithm 1 with `--pc`, on the `_casscf` golden
Offline (`nevpt2_grad.py --case c --reference casscf --pc [--source both --source-path P] --grad-num N`)

1: Algorithm 7's offline lines, with Path 2's changes
2: **PASS** iff Eq. (33) holds against `grad_num_pc`, every check passes, and the guard ledger balances

The equations are Path 3's with Path 2's substitutions. Its numerical gradient is the GPU's PC energy, differenced over the same re-optimised states the SC sweep kept, so the oracle's PC energies come from the card while its states come from PySCF. The case is acrolein CAS(6,5), block2-sourced and GPU-sourced (`docs/testing.md`, "PC on a CASSCF reference: acrolein").

13 Checks that read no residual

A converged residual says that a solve found the answer to the equations it was given, not that those were the right equations. Every solve and every difference quotient in `nevpt2_grad.py` therefore also gets a check that reads no residual. Each such check runs through `Functional.evaluate`, the route Φ is differentiated through, rather than through the code under test. Most are *step ladders*: central differences at $h = 2^{-k}$ over a range of k, gated on two facts.

1. **The ladder converges.** The change between neighbouring rungs falls by a factor in $(3.2, 4.8)$ for at least three consecutive halvings, and of several such runs the finest is used. This is the C++ checks' rule too, `nevpt2::longestFourfoldRun`.

2. **It converges to the right number.** The Richardson extrapolant at that run's finest pair is within `LADDER_TOL` $= 10^{-6}$ of the quantity under test. Comparing an extrapolant, rather than the smallest $|f_d - \mathrm{ref}|$ in the sweep, is what keeps a crossing of a wrong reference from reading as agreement.

check	ladder of	converges to
projection of y, of z	—	$\lvert c \cdot y\rvert$, $\lvert c \cdot z\rvert \le 10^{-6}$
y linearity (two seeds)	E_{corr} along a spin-adapted, in-sector $u \perp c$, on the sphere	$\langle u, y\rangle$
CI stationarity	Eq. (25)'s CI part along u (whole, or by pieces)	0
orbital stationarity	L along a totally symmetric non-gauge κ, $Ce^{t\kappa}$	0
nuclear ladder	Φ along a totally symmetric displacement	$\langle d, g\rangle$
PC guard ledger	—	every displaced c guarded, no rank change
Hessian-vector (CASSCF)	`gen_g_hop`'s g along seeded random v	Jv

Table 4: The residual-independent checks (`check_*` in `nevpt2_grad.py`). The non-gauge rotations of the orbital-stationarity check are derived from the RHF occupied projector, independently of the solve's `kappa_pairs`, so the check does not inherit that bookkeeping. Which check catches which of the errors found so far, with the numbers, is `docs/testing.md`, "Residual-independent checks of the gradient path".

Validation with E_{corr} dropped. `--casci-only` runs the identical pipeline with E_{corr} removed from the source terms and from Φ. On the CASCI branch the reference is then a plain CASCI, and its gradient is gated against `nevpt2_numgrad.py`'s `grad_casci`. That one comparison exercises the frozen path, the full constraint Jacobian, the selection-boundary multipliers and the CPHF block. It does not exercise the CI Z-vector (a CASCI is variational in c, so $y = z = 0$) or Eqs. 56–57, whose numerators vanish for E_{CAS}. `pyscf.grad.casci` is an oracle only for an energy-ordered active space (`--energy-ordered`). On the CASSCF branch the reference becomes a CASSCF, stationary in κ and c, so the multipliers must come out *exactly* zero and the gradient must be `pyscf.grad.casscf`'s (`check_multipliers_vanish`).

The gauge identity (`docs/gradient-theory.md`, §1.3) is reported, not gated: the Lagrangian's derivative along the gauge rotations. On the CASCI branch the constraints' own gauge response is not part of the sum, because f's active sum is not covariant at fixed $\Gamma^{(1)}$. On the CASSCF branch the constraints are covariant and the identity is the whole Lagrangian's.

14 Scope and limits

- **No nuclear derivative in `src/` or `apps/`.** The GPU computes and writes the source terms. The Z-vector solve, the skeleton derivative and the numerical gradient are offline Python. The in-tree Lagrangian gate, `requireLagrangianReference`, refuses a `casscf` golden, and no path in `src/` sits behind it yet (`docs/gradients.md`, §1).
- **The skeleton derivative is differenced, not contracted.** An AO contraction against derivative integrals would need the NEVPT2 relaxed densities: Y fixes only the antisymmetric part of the generalised Fock. `src/gradient/` builds the pseudodensities and frees them as the slab walk ends, and no flag writes them out.
- **The CASCI branch's selection is a discrete function of geometry.** Where it changes, the energy has a kink. A small gap across an active-space boundary makes the selection multipliers large. Validation therefore uses in-plane displacements of planar C_s molecules, or a C_1 one on the CASSCF branch (`docs/gradient-theory.md`, §4). On the CASSCF branch the analogue is the conditioning of J, and a displaced optimisation that lands on another branch.
- **SC's non-invariance among degenerate inactive orbitals** (Park) reaches the gradient through the (O2) divisions on both branches. N2 is the deliberate probe of it, and degenerate pairs follow the rule of §6.
- **PC can refuse.** PC refuses a state whose metric is not gapped, with LiF as the standing example, and a displaced CI vector that lifts a null mode past the cut.
- **Density fitting is SC, CASCI branch, one case.** There is no fitted PC dump and no fitted CASSCF `grad_num`.
- **The second derivative is elsewhere.** The energy's parameter Hessian is in tree; the nuclear Hessian is not. See `docs/hessian-theory.md` and `docs/gradients.md`, §1.1.
- **Cost.** The offline cost is the κ sweep, $O(n_{\text{mo}}^2)$ energy evaluations, and that is the part the GPU's dump replaces for Y. The walls are `docs/reference-data.md`, "The analytic gradient", and `docs/testing.md`, "PC on a CASSCF reference: acrolein".

A The source-term inventory

Y reads, besides what the energy reads, the arrays the promotion rule generates: every stored block with one index replaced by a general one x, keeping the charge-distribution pairing (`docs/gradient-theory.md`, §3.1–3.3). Write $(xP|QR)$ for the array in which x shares a charge distribution with a label from space P. Seventeen of the eighteen are needed, and the sidecar carries them:

array	shape	differentiates
$(xa\|aa)$	$n_{\mathrm{mo}} n_a^3$	$aaaa$, $aaav$, $aaac$
$(xv\|aa)$	$n_{\mathrm{mo}} n_v n_a^2$	$aaav$, $caav$, ε_r
$(xa\|va)$	$n_{\mathrm{mo}} n_a^2 n_v$	$aaav$, $aavv$, $caav$, ε
$(xc\|aa)$	$n_{\mathrm{mo}} n_c n_a^2$	$aaac$, $caav$, h^{eff}, ε_i
$(xa\|ca)$	$n_{\mathrm{mo}} n_a^2 n_c$	$aaac$, $ccaa$, $caav$, ε
$(xv\|va)$	$n_{\mathrm{mo}} n_v^2 n_a$	$aavv$, $cavv$
$(xc\|ca)$	$n_{\mathrm{mo}} n_c^2 n_a$	$ccaa$, $ccav$, h^{eff}
$(xa\|cv)$	$n_{\mathrm{mo}} n_a n_c n_v$	$caav$, $ccav$, $cavv$
$(xv\|ca)$	$n_{\mathrm{mo}} n_v n_c n_a$	$caav$, $ccav$
$(xc\|va)$	$n_{\mathrm{mo}} n_c n_v n_a$	$caav$, $cavv$, h^{eff}
$(xc\|cv)$	$n_{\mathrm{mo}} n_c^2 n_v$	$ccav$, $ccvv$, h^{eff}
$(xv\|cv)$	$n_{\mathrm{mo}} n_v^2 n_c$	$cavv$, $ccvv$
$(xa\|cc)$	$n_{\mathrm{mo}} n_a n_c^2$	h^{eff} (exchange), ε_i
$(xv\|cc)$	$n_{\mathrm{mo}} n_v n_c^2$	h^{eff} (exchange), ε_r
$(xc\|cc)$, two core labels tied	$n_{\mathrm{mo}} n_c^2$	h^{eff}, ε_i
$(xa\|vv)$, virtual labels tied	$n_{\mathrm{mo}} n_a n_v$	ε_r
$(xc\|vv)$, virtual labels tied	$n_{\mathrm{mo}} n_c n_v$	ε_r

Table 5: The one-general-index arrays (`docs/gradient-theory.md`, §3.3). The eighteenth, $(xv|vv)$, is never needed: no integral the energy reads has three virtual labels, and promotion cannot add one.

Besides the integrals, the sidecar carries the full-range h, h^{eff} and its variants, f, f^{h}, `mo_coeff` and, for the CASCI branch only, the RHF Fock F and the RHF electron count (`docs/gradient-theory.md`, §3.5). Three invariants are checked on it. F is block-diagonal over core/active/virtual ((O1); CASCI only). f's core and virtual off-diagonals vanish, and its diagonal is `e_core`, `e_virt` ((O2)). F's active block splits into the right number of occupied and virtual eigenvalues (CASCI only). Density-fitted, each array is $\sum_L B^L_{xp} B^L_{qr}$ (Eq. (34)). It reads all six space blocks of B, and the `NEVPT2F1` sidecar adds the two that a `--df` golden lacks, B_{cc} and B_{vv} (`docs/gradient-theory.md`, §3.8).

B The pseudocanonical multipliers z^c

The energy reads f only on its core and virtual diagonals, so f's off-diagonal response there must be pinned by the convention (O2). In the pseudocanonical gauge f is diagonal on both spaces, so for a rotation inside one of them Eq. (1) collapses to $\delta f_{pq} = \kappa_{qp}(\varepsilon_q - \varepsilon_p)$. The (O2) rows of the stationarity equations then reduce to one division per pair, Park's Eqs. 56–57:

$$z^c_{ij} = \frac{1}{2}\frac{Y_{ij} - Y_{ji}}{\varepsilon_i - \varepsilon_j} \quad (i \neq j \text{ core}), \qquad z^c_{rs} = \frac{1}{2}\frac{Y_{rs} - Y_{sr}}{\varepsilon_r - \varepsilon_s} \quad (r \neq s \text{ virtual}). \tag{35}$$

The code does not divide by hand. It solves (O2)'s rows of M (Eqs. (27) and (30)), whose diagonal holds these denominators, and the degenerate-pair rule of §6 replaces a division the state does not

determine. Neither branch adds anything to this appendix, which is why `docs/gradient-theory.md`, §2.3 calls it shared. The GPU divides the same two blocks of its own Y in `nevpt2_sc_pseudodensity --kappa-full`, where it is a check on Y rather than an input to a gradient (`docs/gradients.md`, §6).